



A

Technical Seminar Report on

"Trie Data Structure and Its Applications in C++"

B.Tech I year II Semester

Submitted by

258810559

B.Sanjana

258810508

Basa Dhruthi

25881A0507

Dhaarmik Reddy

Under the guidance of

Dr. V.Lokeshwari Vinya

Associate Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering,

Hyderabad

April 2026

2025-2026

Table of Contents

1. Introduction
2. Overview of the Topic
3. Key Concepts and Technologies
4. Applications and Use Cases
5. Advantages and Challenges
6. Recent Trends or Future Scope
7. Conclusion
8. References

Abstract

1. **Introduction**
2. **Overview of the Topic**
3. **Key Concepts and Technologies**
4. **Applications and Use Cases**
5. **Advantages and Challenges**
6. **Recent Trends or Future Scope**
7. **Conclusion**

PPT Slides:

Tries: Unlocking Efficient String Operations in C++

A deep dive into the prefix tree — its structure, C++ implementations, real-world applications, and future potential.

DATA STRUCTURES

C++

STRING ALGORITHMS



What is a Trie? The Prefix Tree Explained



A **Trie** (pronounced "try"), also known as a **prefix tree**, is a specialized tree-based data structure designed for efficient storage and retrieval of strings.

Node = Character

Each node represents a single character. Paths from root to any node form a prefix.

End-of-Word Flag

A boolean flag marks nodes that complete a valid word.

Prefix Power

Enables ultra-fast prefix searches, autocomplete, and spell checking.

Key Concepts & C++ Implementations



Insert

Traverse/create nodes, mark isEndOfWord

Search

Follow character path, check end flag

Delete

Remove nodes, handle shared prefixes

These three operations form the backbone of Trie functionality — each running in $O(L)$ time where L is the key length.

Popular C++ Trie Libraries

Arasoul/TrieDataStructure

Modern C++ for ultra-fast search and autocomplete

hedzr/trie-cxx

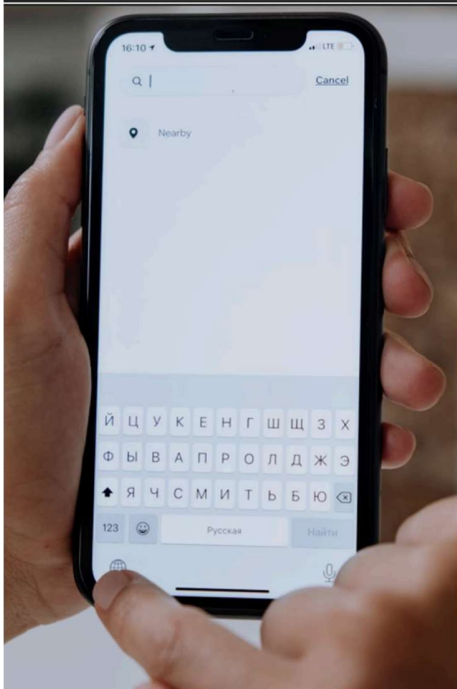
C++17/C++20 header-only with key-value storage

Galfurian/ctrie

Lightweight concurrent Trie for thread-safe ops

Tessil/hat-trie

Fast, memory-efficient HAT-trie implementation



Applications & Use Cases: Where Tries Shine



Autocomplete & Predictive Text

Suggesting words as users type — powering search engines and mobile keyboards.



Spell Checkers

Verifying word existence and generating correction suggestions efficiently.



IP Routing Tables

Managing IP address prefixes for fast, scalable network routing decisions.



Dictionary & Text Search

Compactly storing large dictionaries and indexing text for rapid lookup.

Advantages & Challenges

✓ Advantages

- **$O(L)$ Operations** – Search, insert, and delete scale with key length, not dataset size.
- **Prefix Matching** – Native support for prefix queries without scanning the full dataset.
- **Space Efficiency** – Shared prefixes reduce redundancy in dense string datasets.

⚠ Challenges

Memory Overhead

Can consume more memory than hash tables for sparse datasets with few common prefixes.

Implementation Complexity

More intricate to implement from scratch compared to simpler structures like arrays or hash maps.

Future Scope & Conclusion

🔮 Emerging Trends

01

Concurrent Tries

Libraries like `ctrie` enable thread-safe operations for multi-threaded systems.

02

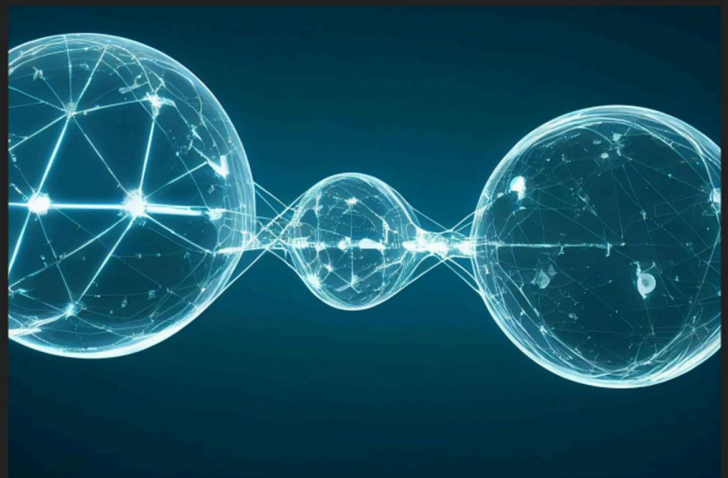
Memory-Optimized Variants

HAT-trie and poplar-trie push the boundaries of space efficiency.

03

ML Integration

Combining Tries with ML for advanced predictive models and edit-distance searches.



- ❏ **Conclusion:** Tries are a powerful, versatile structure for string-intensive C++ applications. Their prefix-operation efficiency makes them indispensable in modern software engineering.

8. References

1. Introduction

In computer science, efficient storage and retrieval of strings are fundamental problems. Traditional data structures like arrays and linked lists are inefficient for string searching because they require comparing entire strings one by one. The Trie data structure, invented by Edward Fredkin in 1960, solves this problem by storing strings in a tree where each path from the root to a leaf represents a complete word.

The name "Trie" comes from the word "retrieval" because it was designed for fast retrieval of strings. A Trie with 26 children per node (one for each letter of the alphabet) can store an entire dictionary and search for any word in $O(L)$ time, where L is the length of the word, regardless of how many words are stored. This makes it significantly faster than arrays or binary search trees for string operations.

In the context of C++ and Data Structures, a Trie is implemented using structs with pointer arrays, making it a perfect example of combining trees, arrays, and pointers in a single data structure. Understanding Trie helps students connect multiple DSS concepts together in one practical and powerful application.

2. Overview of the Topic

A Trie is a tree-like data structure where each node contains an array of 26 pointers (for letters a to z) and a Boolean flag that marks whether that node represents the end of a valid word. The root node is empty and each path from root to a marked node spells out a complete word stored in the Trie.

Structure of a Trie Node in C++:

```
struct TrieNode {  
  
    TrieNode* children[26];  
  
    bool isEndOfWord;  
  
    TrieNode() {  
  
        isEndOfWord = false;  
  
        for(int i = 0; i < 26; i++)  
  
            children[i] = NULL;  
  
    }  
};
```

Example — Storing words CAT, CAR, BAT, BAR:

ROOT

```
  /  \  
 C   B  
 |   |  
 A   A
```


/\ /\

T R T R

END END END END

Each path $ROOT \rightarrow C \rightarrow A \rightarrow T$ represents the word "CAT". The nodes C and A are shared between CAT and CAR, saving memory.

Key Properties:

- Root node is always empty
- Each node has maximum 26 children
- Path from root to any END node = a valid word
- Common prefixes are stored only once
- Search time depends only on word length, not dictionary size

3. Key Concepts and Technologies

3.1 Insert Operation To insert a word, start from root and go letter by letter. If the child node for that letter exists, move to it. If not, create a new node. After inserting the last letter, mark that node as end of word.

```
void insert(TrieNode* root, string word) {  
    TrieNode* curr = root;  
    for(char c : word) {  
        int index = c - 'a';  
        if(!curr->children[index])  
            curr->children[index] = new TrieNode();  
        curr = curr->children[index];  
    }  
    curr->isEndOfWord = true;  
}
```

3.2 Search Operation To search a word, traverse the trie letter by letter. If any letter's child is NULL, word does not exist. If all letters found and last node is marked end, word exists.

```
bool search(TrieNode* root, string word) {  
    TrieNode* curr = root;  
    for(char c : word) {
```

```

    int index = c - 'a';
    if(!curr->children[index])
        return false;
    curr = curr->children[index];
}
return curr->isEndOfWord;
}

```

3.3 Autocomplete Operation Given a prefix like "ca", find the node at end of prefix, then print all words reachable from that node using DFS traversal.

3.4 C++ Concepts Used

- Structs and Classes
- Pointer arrays
- Dynamic memory allocation (new keyword)
- Recursion
- String traversal using loops

3.5 DSS Concepts Used

- Tree data structure
- Array of pointers
- Depth First Search (DFS)
- Insertion and Searching algorithms

4.Applications and Use Cases

4.1 Search Engine Autocomplete When you type "go" in Google, it instantly shows "google", "good morning", "go pro". Google uses a Trie to store billions of search queries and retrieve matching suggestions in milliseconds.

4.2 Phone Keyboard Word Suggestions When you type on your mobile keyboard and it suggests the next word; a Trie is used behind the scenes to find all words matching the letters you typed so far.

4.3 Spell Checker Applications like Microsoft Word and Grammarly use Tries to store the entire dictionary. When you type a word, it searches for the Trie to check if the word exists. If not found, it flags it as a spelling error.

4.4 Contact Search in Phone When you type a name in your phone's contact search, a Trie helps find all contacts whose names start with the typed letters instantly, even if you have thousands of contacts.

4.5 IP Routing in Networks Routers use a special version of Trie called Patricia Trie to store IP addresses and find the best route for data packets in computer networks.

4.6 Word Games and Puzzles Games like Scrabble and Boggle use Tries to validate if a combination of letters forms a valid word from the dictionary, making word validation instant.

4.7 Password and Username Validation Websites maintain a list of banned or common passwords in a Trie. When a user sets a password, it is searched in the Trie to check if it is too common or already banned.

4. Advantages and Challenges

ADVANTAGES

- **Super Fast Search** — Search time is $O(L)$ where L is word length, independent of how many words are stored
- **Efficient Prefix Matching** — Finding all words with a common prefix is extremely fast, making autocomplete easy
- **Shared Storage** — Words with common prefixes share nodes, saving significant memory compared to storing full strings separately
- **No Hash Collisions** — Unlike hash maps, Tries never have collision problems
- **Alphabetical Order** — Words are naturally stored in alphabetical order in a Trie, making sorted output easy
- **Predictable Performance** — Time complexity is always consistent regardless of data size

Challenges:

- **High Memory Usage** — Each node stores 26 pointers even if most are NULL, wasting memory for sparse datasets
- **Complex Implementation** — More complex to implement compared to arrays or linked lists, especially deletion
- **Only for Strings** — Trie is specifically designed for string data and cannot be used for other data types directly
- **Cache Inefficiency** — Pointer-based structure causes frequent cache misses, making it slower in practice than theoretical analysis suggests
- **Deletion is Tricky** — Removing a word requires careful handling to avoid deleting nodes shared with other words

6.Recent Trends or Future Scope

Recent Trends:

- **Compressed Tries (Patricia Trie)** — Modern implementations compress chains of single-child nodes into one edge to save memory. Used in network routers and IP address matching
- **Ternary Search Trie** — A space-efficient variant that uses only 3 pointers per node instead of 26, reducing memory while maintaining fast search
- **Trie in Natural Language Processing** — Large Language Models like ChatGPT and Google Bard use Trie-like structures internally for tokenization, vocabulary storage, and fast text lookup
- **Trie in Databases** — Modern databases like Redis and MongoDB use Trie variants for indexing string keys and performing fast prefix queries
- **Concurrent Tries** — Research is ongoing on thread-safe Trie implementations for multi-core processors to handle simultaneous insertions and searches

Future Scope:

- Tries will continue to grow in importance as the volume of text data on the internet increases exponentially
- DNA sequence analysis in bioinformatics uses Trie structures to search for gene patterns in massive genomic databases
- Cybersecurity applications use Tries for fast malware signature matching and intrusion detection systems
- Voice assistants like Alexa and Siri use Trie-based structures for fast speech-to-text vocabulary matching
- Quantum computing research is exploring quantum versions of Trie for even faster string retrieval

7. CONCLUSION

The Trie data structure is one of the most elegant and practically useful data structures in computer science. By storing strings of character by character in a tree format, it achieves search and insertion in $O(L)$ time, making it far superior to arrays and linked lists for string-heavy applications. Its real-world applications ranging from Google search autocomplete to phone keyboards to network routing make it a highly relevant topic for students learning data structures.

In C++, the Trie is implemented using structs, pointer arrays, and dynamic memory allocation, combining multiple fundamental programming concepts into one powerful structure. Despite its memory challenges, ongoing research into compressed and ternary variants continues to make it more efficient. Understanding Trie not only strengthens knowledge of trees and arrays but also opens doors to advanced topics like compiler design, database indexing, and natural language processing. The Trie data structure truly demonstrates how a simple idea — storing strings letter by letter — can solve some of the most complex problems in modern computing.

8. REFERENCES

- [1] Fredkin, E. (1960). Trie Memory. *Communications of the ACM*, 3(9), 490–499.
- [2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- [3] Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley.
- [4] GeeksforGeeks. (2024). Trie Data Structure. <https://www.geeksforgeeks.org/trie-insert-and-search>
- [5] cppreference.com. (2024). C++ Pointers and Structs. <https://en.cppreference.com/w/cpp>
- [6] Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.

